PYTHON

(sanjay chauhan)

1. **Explain the difference between lists, tuples, sets, and dictionaries.**

In Python, **lists**, **tuples**, **sets**, and **dictionaries** are all used to store collections of data, but they differ in **mutability**, **order**, **duplication**, and **key/value behavior**.

List

Ordered , Mutable (can change), Allows duplicates
Syntax: `my_list = [1, 2, 3]`

I use lists when I want to maintain order and may need to update or access items by index.

Tuple

Ordered, Immutable (cannot change after creation), Allows duplicates
Syntax: `my_tuple = (1, 2, 3)`

I use tuples when I want to protect data from changes – like fixed coordinates or function returns.

Set

Unordered, Mutable, No duplicates allowed
Syntax: `my_set = {1, 2, 3}`

Sets are useful when I need unique values and fast membership tests, like removing duplicates from a list.

Dictionary

- Unordered (ordered in Python 3.7+), Mutable,Stores data as key-value pairs

- Syntax: `my_dict = {"name": "Sanjay", "age": 24}`

  I use dictionaries when I want to map one piece of data to another – like storing user details or counting frequencies.

2. Explain the difference between is and == in Python.

n Python, `==` checks if **two variables have the same value**, while `it checks` if they point to the **same object in memory**.

`==` (Equality Operator)

- Checks for value equality
- Are the contents the same?
- a = [1, 2, 3]
- b = [1, 2, 3]
- print(a == b) # ✅ True – because values are same

`is` (Identity Operator)

- Checks for object identity
- Are `a` and `b` the same object in memory?
- print(a is b) # ❌ False – different objects, even if values are same

If I write `a = [1, 2, 3]` and `b = a`, then `a is b` is `true` because both refer to the same list.

But if I write `b = [1, 2, 3]`, then `a == b` is `True`, but `a is b` is `False` – they are two different lists with the same content.

3. What are Python's different types of loops? How do break, continue, and pass work?

for loop – Iterates over a sequence (like list, tuple, string).
while loop – Repeats as long as a condition is True.

break → Exits the loop **immediately**.
continue → Skips the **current iteration**, moves to the next.
pass → Does **nothing**, acts as a placeholder.

4. What are lambda functions?

A **lambda function** is an **anonymous**, one-line function used for **simple operations**.

```
lambda arguments: expression
```

5. Explain list comprehensions with an example.

A **list comprehension** is a concise way to **create lists** in a single line using a for loop and optional condition.

```
[expression for item in iterable if condition]
```

```
squares = [x**2 for x in range(5)]

print(squares)  # Output: [0, 1, 4, 9, 16]
```

6. What is the difference between map(), filter(), and reduce()?

map():Apply a Function to Every Element

Purpose: Applies a function to **each item**

Returns: A new iterable of results

filter():Select Elements Based on a Condition

Purpose: Selects items based on **condition**

Returns: A new iterable with filtered items

reduce():Reduce a List to a Single Value

Purpose: **Combines** items into a single value

Returns: A single result

```
from functools import reduce

nums = [1, 2, 3, 4]

# map: square each number

print(list(map(lambda x: x**2, nums)))  # [1, 4, 9, 16]

# filter: keep even numbers

print(list(filter(lambda x: x % 2 == 0, nums)))  # [2, 4]

# reduce: multiply all numbers

print(reduce(lambda x, y: x * y, nums))  # 24
```

7. **How does exception handling work in Python? Explain try, except, finally, and raise.**

Python uses `try`, `except`, `finally`, and `raise` to handle errors **gracefully** without crashing the program.

try → Code that might cause an error
except → Code that runs **if an error occurs**
finally → Runs **no matter what** (used for cleanup)
raise → Manually **raise an exception**

```
try:
    x = 1 / 0
except ZeroDivisionError:
```

```
    print("Cannot divide by zero!")
finally:
    print("This will always run.")
x = -5
if x < 0:
    raise ValueError("x cannot be negative")
```

8. What is the difference between loc[] and iloc[]?

In Pandas, `loc[]` and `iloc[]` are used to access rows and columns, but they differ in **how** they select data.

`loc[]` → Label-based indexing

Uses **row and column labels**.
Includes the **end label**.

```
df.loc[1:3, 'name']  # Rows with labels 1 to 3, column 'name'
```

`iloc[]` → Integer-based indexing

Uses **row and column positions** (like arrays).
Excludes the **end index**.

```
df.iloc[1:3, 0]  # Rows 1 and 2, first column
```

9. Explain the difference between apply(), map(), and applymap().

`apply()` – Applies a Function to Rows/Columns in a DataFrame

Works on both DataFrames and Series
Can apply functions **row-wise** (`axis=1`) or **column-wise** (`axis=0`)

Example: Applying a Function to a Column (Series)

```
import pandas as pd
# Creating a DataFrame
data = {'A': [1, 2, 3], 'B': [4, 5, 6]}
df = pd.DataFrame(data)
# Applying a function to double each value in column
'A'
df['A'] = df['A'].apply(lambda x: x * 2)
print(df)
```

Example: Applying a Function to Each Row

```
df['Sum'] = df.apply(lambda row: row['A'] +
row['B'], axis=1)

print(df)
```

Use `apply()` when you need to apply a function row-wise or column-wise.

`map()` – Element-wise Function for Series

🔹 Works only on Pandas Series (single column)
🔹 Can apply a function, dictionary, or another Series

```
df['A'] = df['A'].map(lambda x: x * 3)

print(df)
```

`applymap()` – Element-wise Function for DataFrames

🔹 **Works on an entire DataFrame (applies function to every element)**

```
df_numeric = df[['B', 'Sum']]  # Selecting numeric
columns
```

```
df_numeric = df_numeric.applymap(lambda x: x * 2)

print(df_numeric)
```

10. What is the difference between merge(), join(), and concat() in Pandas?

All three are used to **combine DataFrames**, but they differ in **how** they do it and **what they match on**.

🔹 merge() – SQL-style joins

- Joins DataFrames using **columns or indexes**.
- Supports **inner, outer, left, right** joins.

```
pd.merge(df1, df2, on='id', how='inner')
```

🔹 join() – Shortcut for joining on index

- Works like `merge()` but joins **on index by default**.
- Simpler syntax when joining by index.

```
df1.join(df2, how='left')
```

concat() – Stacks DataFrames

- **Concatenates** along rows (`axis=0`) or columns (`axis=1`).
- Doesn’t match values – just stacks.

```
pd.concat([df1, df2], axis=0)
```